

Investigación Generación de Diagramas funcionales y Arquitectura de Software

Ricardo Andrés Torres Silva

Facultad de Ciencias de la Computación, Universidad Internacional del Ecuador

Lógica de programación 1-CIB-1B

Mtr. Mónica Patricia Salazar Tapia

30 de mayo de 2026



Investigación Introductoria: Diagramas Funcionales y Arquitectura de Software

Diagramas Funcionales en UML

En UML (Lenguaje Unificado de Modelado), los diagramas funcionales o de comportamiento se utilizan para describir cómo funciona un sistema, representando las interacciones entre sus elementos y la manera en que responde a distintos eventos. Estos diagramas permiten modelar los requisitos funcionales, las interacciones entre los usuarios y el sistema, así como los flujos de procesos y actividades (Creately, s. f.).

A diferencia de los diagramas estructurales, que muestran la organización y composición del sistema, los diagramas funcionales se enfocan en representar las acciones y comportamientos que el sistema realiza (Creately, s. f.).

Diagramas de Casos de Uso

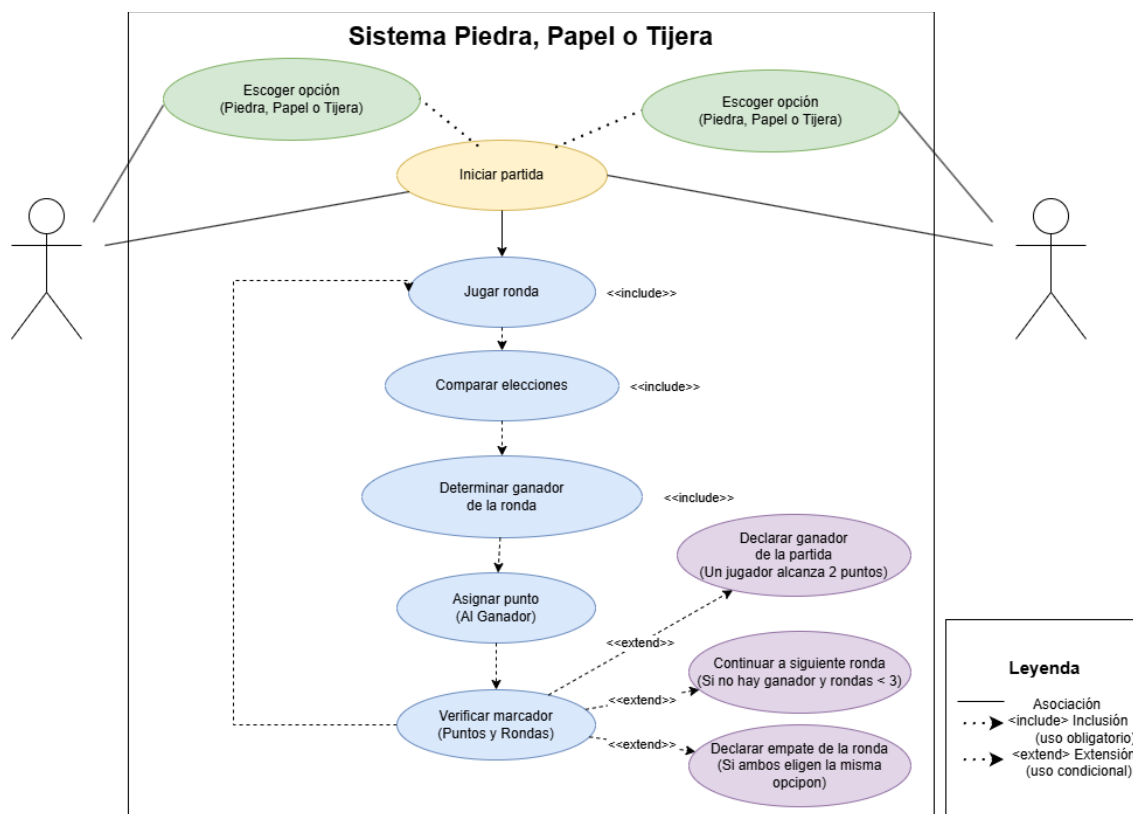
Los diagramas de casos de uso representan las funcionalidades del sistema desde la perspectiva del usuario. Su objetivo es mostrar la interacción entre los actores (usuarios o sistemas externos) y las diferentes funciones que ofrece el sistema. Sus principales elementos son:

- Actor: entidad que interactúa con el sistema, ya sea una persona, organización o sistema externo.
- Caso de uso: función, servicio o tarea que el sistema proporciona.
- Relaciones: conexiones entre actores y casos de uso, incluyendo asociación, inclusión (include) y extensión (extend).

Los diagramas de casos de uso se emplean para definir los requisitos funcionales, comprender qué hace el sistema y facilitar la comunicación entre desarrolladores, clientes y usuarios finales (Creately, s. f.).

Figura 1

Diagrama de caso de uso



Nota: elaboración propia

Diagramas Funcionales en la Arquitectura de Software

La arquitectura de software corresponde a la estructura de alto nivel de un sistema, definiendo sus componentes, relaciones y organización. Para representarla se utilizan diferentes tipos de diagramas que permiten visualizar tanto la construcción del sistema como su

funcionamiento. Estos diagramas facilitan la comprensión del sistema, la comunicación del diseño, la planificación del desarrollo y las actividades de mantenimiento (ProcessOn, s. f.).

Tipos de Diagramas en Arquitectura de Software

La arquitectura de software utiliza principalmente diagramas estructurales y diagramas de comportamiento, entre los cuales destacan:

Diagrama de Componentes. Representa los módulos o componentes que conforman el sistema y las relaciones existentes entre ellos. Incluye interfaces, dependencias y conexiones entre componentes, permitiendo visualizar la organización general de la arquitectura (ProcessOn, s. f.).

Diagrama de Despliegue (Deployment). Muestra cómo los componentes de software se distribuyen e instalan en la infraestructura física, como servidores, dispositivos o entornos en la nube. Representa nodos, redes y conexiones físicas, siendo útil para la planificación de infraestructura y sistemas distribuidos (ProcessOn, s. f.).

Diagrama de Paquetes. Agrupa elementos relacionados en categorías o módulos lógicos. Además, permite representar las dependencias entre paquetes, facilitando la organización del código y la división del sistema en capas o módulos (ProcessOn, s. f.).

Diagramas de Comportamiento. Incluyen diagramas como los de casos de uso, actividades, secuencia y estados. Su propósito es representar las funciones que realiza el sistema y describir los procesos, flujos de trabajo e interacciones entre sus diferentes componentes y usuarios (ProcessOn, s. f.).

Modelo de Arquitectura 4+1

El modelo de arquitectura 4+1, propuesto por Philippe Kruchten, es uno de los enfoques más utilizados para describir arquitecturas de software complejas. Este modelo permite representar un sistema desde diferentes perspectivas, facilitando su comprensión por parte de desarrolladores, arquitectos, usuarios y demás interesados (Kruchten, 1995). La arquitectura se describe mediante cinco vistas complementarias:

1. Vista Lógica

Describe la funcionalidad del sistema y la organización de sus elementos, para satisfacer los requisitos funcionales. Generalmente se representa mediante diagramas de clases y diagramas de casos de uso (Kruchten, 1995). Esta vista se enfoca en qué hace el sistema.

2. Vista de Desarrollo

Muestra la organización del software en módulos, componentes y paquetes. Se representa mediante diagramas de componentes y diagramas de paquetes (Kruchten, 1995). Esta vista se enfoca en la construcción del sistema.

3. Vista de Procesos

Representa el comportamiento dinámico del sistema durante su ejecución, incluyendo procesos, hilos de ejecución y comunicaciones entre componentes. Utiliza diagramas de secuencia, actividad y otros diagramas de comportamiento (Kruchten, 1995). Esta vista se enfoca en cómo funciona el sistema en tiempo de ejecución.

4. Vista Física o de Despliegue

Describe la infraestructura tecnológica donde se ejecuta el sistema, incluyendo servidores, dispositivos, redes y conexiones físicas. Se representa mediante diagramas de despliegue (Kruchten, 1995). Esta vista se enfoca en dónde se ejecuta el sistema.

5. Vista de Escenarios (+1)

Incluye casos de uso y escenarios reales que ilustran la interacción de los usuarios con el sistema. Su propósito es validar y verificar que las demás vistas satisfacen los requisitos establecidos (Kruchten, 1995).

Importancia de los Diagramas en la Arquitectura de Software

Los diagramas constituyen herramientas fundamentales dentro de la arquitectura de software, ya que permiten:

- Visualizar sistemas complejos de manera clara y organizada.
- Facilitar la comunicación entre desarrolladores, analistas, clientes y usuarios.
- Identificar errores y problemas potenciales antes de la implementación.
- Favorecer el mantenimiento y la escalabilidad del sistema.
- Documentar la arquitectura y el diseño del software.
- El uso combinado de diferentes diagramas proporciona una visión integral del sistema, permitiendo analizar tanto su estructura como su comportamiento.

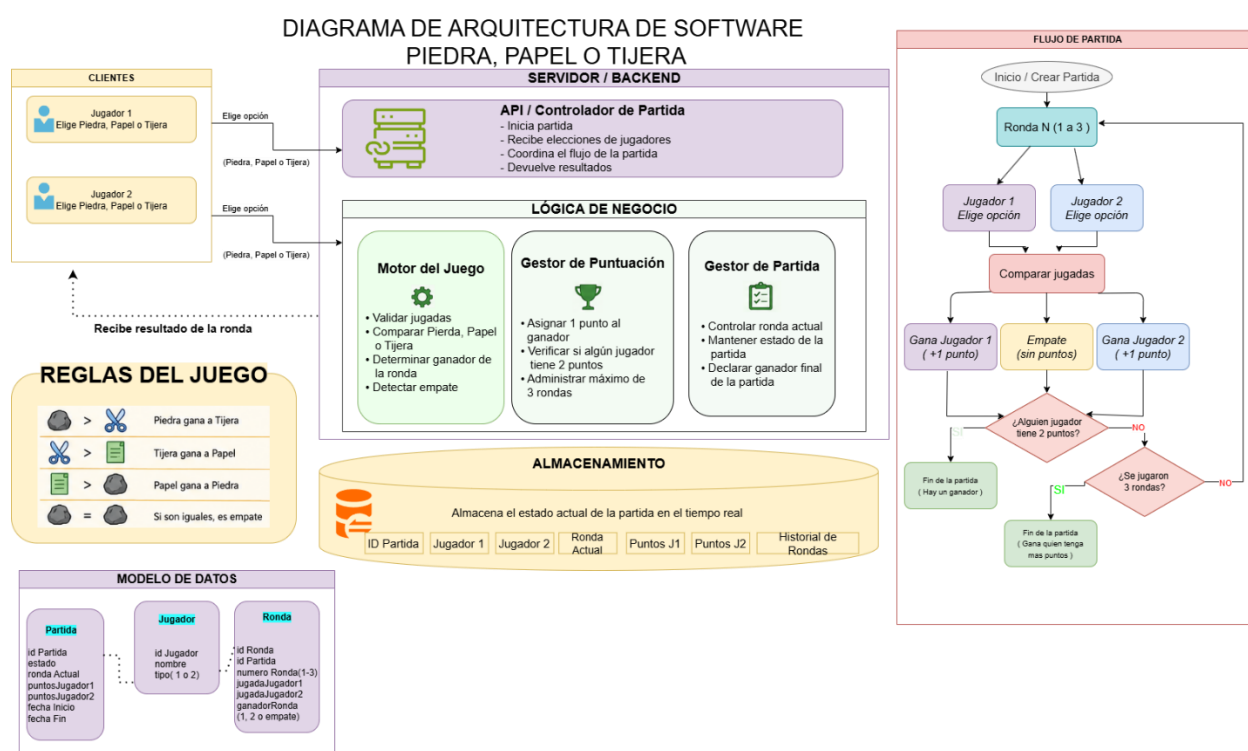
Conclusión

En la arquitectura de software, los diagramas son elementos esenciales para representar la estructura, el comportamiento y la organización de un sistema. El modelo 4+1 integra distintas

perspectivas arquitectónicas que permiten comprender el sistema de forma completa. Entre los diagramas más utilizados destacan los diagramas de componentes, despliegue, clases, secuencia y casos de uso. Gracias a esta metodología, es posible diseñar sistemas más claros, organizados, mantenibles y eficientes.

Figura 2

Diagrama de arquitectura



Nota: Elaboración propia

Bibliografía

Creately. (s. f.). *UML diagram types: Learn about all 14 types of UML diagrams.*

Creately. <https://creately.com/blog/diagrams/uml-diagram-types/>

Kruchten, P. B. (1995). *The 4+1 view model of architecture*. *IEEE Software*, 12(6), 42– 50.

<https://doi.org/10.1109/52.469759>

ProcessOn. (s. f.). *The complete guide to software architecture diagrams: Concepts, tutorials, and examples*. ProcessOn. <https://www.processon.io/>